Wortschatz SRS

Aplicación web de repaso espaciado de vocabulario alemán (A1.2)

Especificación técnica completa para implementación con Claude Code

Campo	Valor
Arquitectura	Monolito modular (una sola aplicación desplegable)
Stack	Next.js (App Router) + TypeScript · Prisma ORM · PostgreSQL 16 · Tailwind CSS
Algoritmo de repaso	FSRS mediante la librería `ts-fsrs`, aislada detrás de una interfaz propia
Despliegue	Docker multi-stage + Docker Compose (app, migraciones, base de datos)
Idioma de la interfaz	Español (contenido de estudio en alemán)
Usuario objetivo	Hispanohablante en nivel A1.2 (Kursbuch/Arbeitsbuch lecciones 13–16 y apuntes de clase)
Versión del documento	1.0 · octubre de 2026

Cómo usar este documento: entregarlo completo a Claude Code junto con la carpeta `materiales/`. La sección 13 define el orden de trabajo por fases y la sección 14 contiene el `CLAUDE.md` que debe copiarse en la raíz del repositorio. Las palabras DEBE / NO DEBE indican requisitos obligatorios; DEBERÍA indica recomendación fuerte.

Contenido

Contenido 2

1. Propósito y alcance 4

1.1 Dentro del alcance (versión 1) . . . 4

1.2 Fuera de alcance (versión 1) . . . 4

2. Fundamentos pedagógicos → requisitos 5

3. Stack tecnológico 6

4. Arquitectura 7

4.1 Monolito modular . . . 7

4.2 Reglas de dependencia . . . 7

4.3 Módulos . . . 7

5. Estructura del repositorio 9

6. Modelo de datos (PostgreSQL + Prisma) 10

6.1 schema.prisma . . . 10

6.2 Reglas de integridad (validadas con Zod en el servicio) . . . 12

7. Lógica de negocio 13

7.1 Tipos de tarjeta por palabra . . . 13

7.2 Prioridad de introducción . . . 13

7.3 Día de estudio y zona horaria . . . 13

7.4 Construcción de la cola diaria (study/queue.ts) . . . 14

7.5 Programación FSRS (srs/) . . . 14

7.6 Corrección de respuestas (study/answer-check.ts) . . . 15

7.7 Registro de un repaso (transacción) . . . 15

7.8 Deshacer . . . 15

7.9 Práctica libre (no afecta FSRS) . . . 15

8. Interfaz y rutas 17

8.1 Páginas . . . 17

8.2 Pantalla de estudio . . . 17

8.3 Diseño . . . 18

8.4 API JSON (Route Handlers) . . . 18

9. Vocabulario: formato, extracción y semilla 19

9.1 Formato CSV (importación, exportación y semilla) . . . 19

9.2 Fase 0 – Extracción del vocabulario desde materiales/ . . . 19

9.3 Semilla (prisma/seed.ts) . . . 20

9.4 Exportaciones . . . 20

10. Docker y entornos 21

10.1 Dockerfile (multi-stage) . . . 21
10.2 docker-compose.yml (producción / uso diario) . . . 21
10.3 docker-compose.dev.yml y docker-compose.test.yml . . . 22
10.4 Variables de entorno (.env.example) . . . 22
10.5 Copias de seguridad de la base de datos . . . 22
10.6 Scripts de package.json . . . 22
11. Seguridad 23
12. Pruebas y criterios de aceptación 24
12.1 Estrategia . . . 24
12.2 Casos obligatorios de checkAnswer . . . 24
12.3 Criterios de aceptación (Dado / Cuando / Entonces) . . . 24
13. Plan de implementación por fases 26
14. CLAUDE.md (copiar en la raíz del repositorio) 27
15. Evolución futura (fuera de la versión 1) 28

1. Propósito y alcance

Wortschatz SRS es una aplicación web personal para aprender vocabulario alemán con dos técnicas de estudio respaldadas por la investigación: **recuperación activa** (el estudiante produce la respuesta antes de verla) y **repaso espaciado** (el sistema programa cada repaso justo antes de que la palabra se olvide). A diferencia de Anki, la aplicación conoce la gramática alemana: exige artículo y plural en los sustantivos, trata los verbos separables e irregulares, colorea el género y marca falsos amigos.

El vocabulario se organiza en tres niveles de prioridad, y la app introduce las palabras nuevas en ese orden:

Nivel	Criterio	Objetivo de aprendizaje
A – alta frecuencia	Aparece en los textos del estudiante y en la lista oficial Goethe A1 o A2.	Uso activo: entender y producir.
B – clave en los textos	Aparece en 2 o más fuentes del estudiante o en recuadros de vocabulario de la lección; ligada al objetivo comunicativo.	Uso activo dentro del tema.
C – específico	Aparece una sola vez o es muy temático; no está en las listas A1/A2.	Solo reconocimiento (alemán → español).

1.1 Dentro del alcance (versión 1)

- Cuentas de usuario con email y contraseña; datos de cada usuario aislados.
- Catálogo global de vocabulario cargado desde un CSV (semilla) y palabras propias por usuario.
- Sesión de estudio con repaso espaciado FSRS, varios tipos de tarjeta y corrección automática de respuestas escritas.
- Reglas de carga diaria: límite de palabras nuevas, tope de repasos y bloqueo de nuevas si hay acumulación.
- Práctica libre sin afectar la programación: drill de der/die/das y repaso por lección.
- Palabras rebeldes (leeches) con espacio para mnemotecnia y frase propia.
- Estadísticas: racha, repasos por día, retención, previsión, progreso por nivel y lección.
- Importación/exportación CSV, exportación compatible con Anki y copia de seguridad JSON.
- Pronunciación con la API de voz del navegador (de-DE).
- Despliegue completo con `docker compose up --build`.

1.2 Fuera de alcance (versión 1)

- Modo offline / PWA, aplicaciones nativas, audio grabado por humanos, reconocimiento de voz.
- Generación de contenido con IA dentro de la app.
- Microservicios, colas de mensajes, caché externa (Redis) o escalado horizontal.
- Otros idiomas distintos del alemán.

2. Fundamentos pedagógicos → requisitos

Cada decisión funcional se deriva de un principio de aprendizaje. Esta tabla es la trazabilidad que Claude Code DEBE respetar; si una implementación contradice un principio, se considera un defecto.

Principio	Requisito en la app
Recuperación activa	Las tarjetas de producción (español → alemán) exigen escribir la respuesta antes de revelarla. Revelar sin escribir solo está permitido en tarjetas de reconocimiento.
Repaso espaciado	Programación con FSRS, retención deseada configurable (por defecto 0,90), fuzz activado, intervalo máximo 365 días.
Frecuencia primero	Orden de introducción por prioridad: A★ → A → B (por lección) → C.
Carga sostenible	12 palabras nuevas/día por defecto (rango 5–30); si los repasos pendientes superan 120, no se introducen nuevas.
Sustantivo = artículo + plural	Un sustantivo NO DEBE guardarse sin género. El plural es obligatorio salvo marca explícita «sin plural».
Género visual	Colores fijos: der azul #2563EB, die rojo #DC2626, das verde #16A34A, plural gris #6B7280.
Contexto	Cada palabra DEBERÍA tener frase de ejemplo; las expresiones se estudian como tarjeta cloze.
Falsos amigos	Campo booleano + nota; se muestra un aviso visible y suben de prioridad.
Mnemotecnia solo para rebeldes	El campo de mnemotecnia se ofrece de forma destacada solo cuando una tarjeta se marca como rebelde (lapses ≥ 6).
Uso activo	Campo «mi frase» por palabra; la app invita a escribirla tras el tercer repaso correcto.
Hermanas separadas	Dos tarjetas de la misma palabra NO DEBEN aparecer el mismo día de estudio.

3. Stack tecnológico

Se elige un stack TypeScript de punta a punta para que la interfaz, la lógica y el acceso a datos vivan en un único proceso Node.js: es la forma más simple de cumplir la arquitectura monolítica.

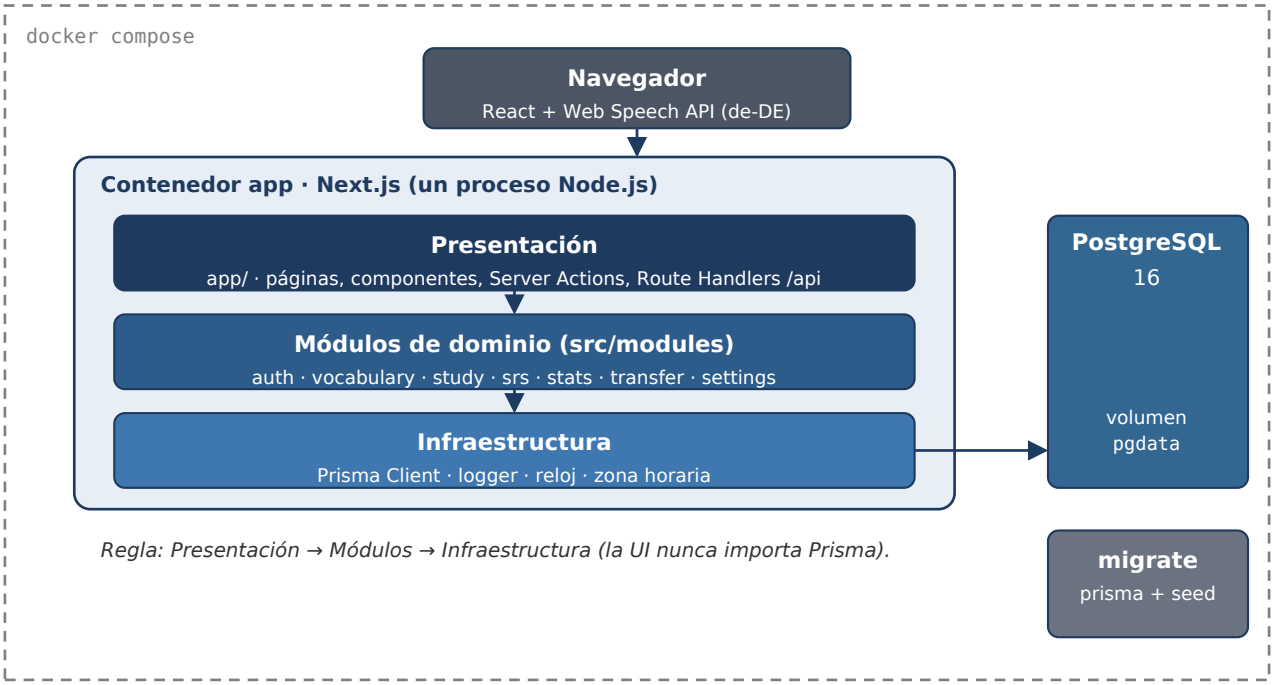
Capa	Tecnología	Motivo
Runtime	Node.js 22 LTS	Versión LTS estable; imagen oficial `node:22-alpine`.
Framework	Next.js 15 (App Router), React 19	UI, Server Components, Server Actions y Route Handlers en un solo proceso. `output: "standalone"` para Docker.
Lenguaje	TypeScript 5 (`strict: true`)	Tipos compartidos entre UI, servicios y base de datos.
Base de datos	PostgreSQL 16	Requisito del proyecto; arrays, enums y transacciones.
ORM / migraciones	Prisma 6	Esquema declarativo, migraciones versionadas, cliente tipado.
Repaso espaciado	`ts-fsrs` (open-spaced-repetition)	Implementación mantenida de FSRS; evita escribir el algoritmo a mano.
Validación	Zod	Validar entradas de Server Actions, API y CSV.
Estilos / UI	Tailwind CSS 4 + shadcn/ui (Radix) + lucide-react	Componentes accesibles, modo oscuro.
Gráficos	Recharts	Estadísticas y previsión.
Fechas / zonas horarias	date-fns 4 + `@date-fns/tz`	Día de estudio en la zona horaria del usuario.
Contraseñas	`@node-rs/argon2` (Argon2id)	Hash moderno sin dependencias nativas problemáticas en Alpine.
CSV	PapaParse	Importación/exportación robusta con UTF-8.
Logs	pino	Logs JSON en stdout (los recoge Docker).
Tests	Vitest, Testing Library, Playwright	Unitarios, integración y extremo a extremo.
Calidad	ESLint, Prettier, `tsc --noEmit`	Se ejecutan en `pnpm check`.
Paquetes	pnpm (vía corepack)	Instalaciones reproducibles con lockfile.
Contenedores	Docker multi-stage + Docker Compose v2	Requisito del proyecto.

Versiones: las versiones mayores de la tabla son las de referencia. Al iniciar el proyecto, Claude Code DEBE instalar la última versión estable de cada línea indicada, fijarla en el lockfile y comprobar en la documentación instalada la API exacta (especialmente `ts-fsrs`, cuyos nombres de campos han cambiado entre versiones mayores). Si una versión mayor indicada ya no existe o tiene un cambio incompatible, usar la más cercana y anotarlo en `docs/DECISIONES.md`.

4. Arquitectura

4.1 Monolito modular

Una sola aplicación Next.js contiene todo: interfaz, lógica de negocio y acceso a datos. Se despliega como un único contenedor `app` junto a un contenedor `db` (PostgreSQL). Internamente se divide en **módulos de dominio** con fronteras claras, de modo que el código se mantiene ordenado sin la complejidad de microservicios.



4.2 Reglas de dependencia

- Los componentes y páginas NO DEBEN importar `@prisma/client` ni `src/server/db.ts`; solo llaman a servicios (desde Server Components o Server Actions) o a `/api`.
- Cada módulo expone su API pública en `src/modules/<modulo>/index.ts`. Otro módulo NO DEBE importar archivos internos de un módulo distinto.
- La lógica pura (corrección de respuestas, prioridad, cola diaria, rachas) vive en funciones sin E/S, con el reloj inyectado (`now: Date`), para poder probarla sin base de datos.
- Todos los repositorios reciben `userId` como primer argumento y filtran por él. No existe consulta de datos de usuario sin `userId`.
- Las operaciones que tocan varias tablas (registrar un repaso, deshacer, importar) DEBEN ejecutarse en una transacción `prisma.$transaction`.

4.3 Módulos

Módulo	Responsabilidad
`auth`	Registro, login, logout, sesiones en base de datos, middleware de protección de rutas, límite de intentos.
`vocabulary`	Catálogo global y palabras propias: CRUD, filtros, cálculo de prioridad, generación de tarjetas por palabra, notas (mnemotecnia y frase propia).

Módulo	Responsabilidad
`srs`	Adaptador de `ts-fsrs` detrás de la interfaz `Scheduler`; vista previa de intervalos; detección de rebeldes.
`study`	Construcción de la cola diaria, corrección de respuestas, registro de repasos, deshacer, práctica libre.
`stats`	Racha, repasos por día, retención, previsión, progreso por nivel y lección.
`transfer`	Importación CSV, exportación CSV/Anki, copia de seguridad y restauración JSON.
`settings`	Preferencias del usuario con valores por defecto y validación de rangos.

5. Estructura del repositorio

```
wortschatz-srs/
├── CLAUDE.md                    # reglas para Claude Code (sección 14)
├── README.md
├── docs/
│   ├── ESPECIFICACION.pdf       # este documento
│   └── DECISIONES.md            # registro de decisiones (ADR breves)
├── materiales/                  # fuentes del vocabulario (NO se sirven en la web)
│   ├── A1_2_Kursbuch13141516.pdf
│   ├── Arbeitbuch13141516.pdf
│   ├── clases/                  # M3_Clase_*.docx, M4_Clase_*.docx (en realidad Markdown)
│   └── goethe/                  # opcional: Wortlisten oficiales A1/A2 en PDF
├── data/
│   ├── vocabulario.csv          # semilla del catálogo global (formato sección 9)
│   ├── lecciones.csv            # número;título;tema;orden
│   └── REVISION.md              # dudas de la extracción para revisión humana
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts                  # idempotente
├── src/
│   ├── app/
│   │   ├── (auth)/login/page.tsx
│   │   ├── (auth)/registro/page.tsx
│   │   ├── (app)/layout.tsx             # navegación + guardia de sesión
│   │   ├── (app)/page.tsx               # Panel
│   │   ├── (app)/estudiar/page.tsx
│   │   ├── (app)/practicar/page.tsx
│   │   ├── (app)/vocabulario/page.tsx
│   │   ├── (app)/vocabulario/nueva/page.tsx
│   │   ├── (app)/vocabulario/[id]/page.tsx
│   │   ├── (app)/rebeldes/page.tsx
│   │   ├── (app)/estadisticas/page.tsx
│   │   ├── (app)/importar-exportar/page.tsx
│   │   ├── (app)/ajustes/page.tsx
│   │   └── api/                         # Route Handlers JSON (sección 8)
│   ├── components/
│   │   ├── ui/                          # shadcn/ui
│   │   ├── study/                       # StudyCard, AnswerInput, UmlautBar, RatingBar...
│   │   └── word/                        # GenderBadge, WordForm, WordTable...
│   ├── modules/
│   │   ├── auth/      {index.ts, service.ts, repository.ts, session.ts, schemas.ts}
│   │   ├── vocabulary/{index.ts, service.ts, repository.ts, priority.ts, card-kinds.ts, schemas.ts}
│   │   ├── srs/       {index.ts, scheduler.ts, fsrs-scheduler.ts, leech.ts}
│   │   ├── study/     {index.ts, service.ts, queue.ts, answer-check.ts, study-day.ts}
│   │   ├── stats/     {index.ts, service.ts, streak.ts}
│   │   ├── transfer/  {index.ts, csv.ts, anki.ts, backup.ts, schemas.ts}
│   │   └── settings/  {index.ts, service.ts, defaults.ts, schemas.ts}
│   ├── server/
│   │   ├── db.ts                        # singleton de PrismaClient
│   │   ├── logger.ts
│   │   └── clock.ts                     # now() inyectable
│   └── middleware.ts
├── tests/
│   ├── unit/  integration/  e2e/  fixtures/
├── scripts/
│   ├── backup.sh  restore.sh
├── Dockerfile
├── docker-compose.yml
├── docker-compose.dev.yml
├── docker-compose.test.yml
├── .env.example  .dockerignore
├── next.config.ts  tsconfig.json  vitest.config.ts  playwright.config.ts
└── package.json  pnpm-lock.yaml
```

Convenciones: código, nombres de archivos y base de datos en **inglés**; textos de la interfaz, mensajes de error al usuario y documentación en **español**. Las rutas visibles de la web están en español.

6. Modelo de datos (PostgreSQL + Prisma)

El catálogo de palabras es global (`scope = "global"`) y cada usuario puede añadir palabras propias (`scope = <userId>`). Las tarjetas, repasos, notas y ajustes siempre pertenecen a un usuario.

6.1 schema.prisma

```
generator client {
  provider      = "prisma-client-js"
  binaryTargets = ["native", "linux-musl-openssl-3.0.x"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum WordType      { NOUN VERB ADJECTIVE ADVERB PREPOSITION PRONOUN EXPRESSION OTHER }
enum Gender        { DER DIE DAS }
enum Level         { A B C }
enum GoetheLevel   { A1 A2 B1 }
enum Auxiliary     { HABEN SEIN }
enum CardKind      { ES_TO_DE DE_TO_ES PLURAL VERB_FORMS SEPARABLE_ORDER CLOZE }
enum CardState     { NEW LEARNING REVIEW RELEARNING }
enum Rating        { AGAIN HARD GOOD EASY }
enum AnswerVerdict { CORRECT ALMOST WRONG SELF_GRADED }

model User {
  id           String        @id @default(cuid())
  email        String        @unique
  passwordHash String
  displayName  String
  createdAt    DateTime      @default(now())
  sessions     Session[]
  settings     UserSettings?
  cards        Card[]
  reviewLogs   ReviewLog[]
  notes        WordNote[]
}

model Session {
  id        String   @id            // SHA-256 (hex) del token; el token solo vive en la cookie
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  expiresAt DateTime
  createdAt DateTime @default(now())
  @@index([userId])
}

model UserSettings {
  userId              String     @id
  user                User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  newCardsPerDay      Int        @default(12)    // 5..30
  maxReviewsPerDay    Int        @default(200)   // 50..999
  reviewBacklogCap    Int        @default(120)   // si hay más pendientes, 0 nuevas
  desiredRetention    Float      @default(0.9)   // 0.80..0.95
  maximumIntervalDays Int        @default(365)
  dayStartsAtHour     Int        @default(4)     // 0..23, hora local
  timezone            String     @default("America/La_Paz")
  enabledCardKinds    CardKind[] @default([ES_TO_DE, DE_TO_ES, PLURAL, VERB_FORMS, SEPARABLE_ORDER, CLOZE])
  ttsEnabled          Boolean    @default(true)
  ttsAutoplay         Boolean    @default(false)
  ttsRate             Float      @default(0.9)   // 0.5..1.2
  strictCapitalization Boolean   @default(false)
  leechThreshold      Int        @default(6)
  leechAutoSuspend    Boolean    @default(false)
  updatedAt           DateTime   @updatedAt
}

model Lesson {
  id        Int     @id @default(autoincrement())
  number    Int     @unique             // 13, 14, 15, 16 ... o 900+ para apuntes de clase
  title     String
  topic     String?
  sortOrder Int
```

```
  words     Word[]
}

model Word {
  id                String       @id @default(cuid())
  scope             String       @default("global")   // "global" | userId
  lemma             String       // "Bahnhof", "ab|biegen", "Wie komme ich zum ...?"
  type              WordType
  gender            Gender?      // obligatorio si type = NOUN
  pluralSuffix      String?      // "-¨e", "-n", "-" (sin cambio), "-s"
  pluralForm        String?      // "Bahnhöfe"
  noPlural          Boolean      @default(false)
  translationEs     String       // alternativas separadas por " / "
  exampleDe         String?
  exampleEs         String?
  clozeText         String?      // frase con {{...}} marcando el hueco
  level             Level
  starred           Boolean      @default(false)   // ★ = cumple A y B
  goetheLevel       GoetheLevel?
  frequencyRank     Int?
  isCognate         Boolean      @default(false)
  isFalseFriend     Boolean      @default(false)
  falseFriendNote   String?
  separablePrefix   String?      // "ab"
  thirdPerson       String?      // "biegt ab"
  perfekt           String?      // "ist abgebogen"
  auxiliary         Auxiliary?
  isIrregular       Boolean      @default(false)
  governsCase       String?      // "Dat.", "Akk.", "Dat./Akk."
  lessonId          Int?
  lesson            Lesson?      @relation(fields: [lessonId], references: [id])
  sources           String[]     // ["KB13","AB13","CLASE3"]
  tags              String[]
  priority          Int          // calculado (sección 7.2), se recalcula al guardar
  createdAt         DateTime     @default(now())
  updatedAt         DateTime     @updatedAt
  cards             Card[]
  notes             WordNote[]

  @@unique([scope, lemma, type])
  @@index([scope, level, priority])
  @@index([lessonId])
}

model Card {
  id            String     @id @default(cuid())
  userId        String
  user          User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  wordId        String
  word          Word       @relation(fields: [wordId], references: [id], onDelete: Cascade)
  kind          CardKind
  state         CardState  @default(NEW)
  due           DateTime   @default(now())
  stability     Float      @default(0)
  difficulty    Float      @default(0)
  elapsedDays   Int        @default(0)
  scheduledDays Int        @default(0)
  learningSteps Int        @default(0)
  reps          Int        @default(0)
  lapses        Int        @default(0)
  lastReview    DateTime?
  introducedAt  DateTime?  // primera vez que se estudió
  suspended     Boolean    @default(false)
  isLeech       Boolean    @default(false)
  reviewLogs    ReviewLog[]

  @@unique([userId, wordId, kind])
  @@index([userId, suspended, state, due])
}

model ReviewLog {
  id             String        @id @default(cuid())
  userId         String
  user           User          @relation(fields: [userId], references: [id], onDelete: Cascade)
  cardId         String
  card           Card          @relation(fields: [cardId], references: [id], onDelete: Cascade)
  rating         Rating
  verdict        AnswerVerdict
```

```
  typedAnswer    String?
  durationMs     Int
  stateBefore    CardState
  cardBefore     Json          // instantánea completa para "deshacer"
  scheduledDays  Int
  reviewedAt     DateTime      @default(now())
  studyDay       String        // "2026-10-06" en la zona horaria del usuario
  isNewCard      Boolean       // true si esta revisión introdujo la tarjeta

  @@index([userId, reviewedAt])
  @@index([userId, studyDay])
}

model WordNote {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  wordId      String
  word        Word     @relation(fields: [wordId], references: [id], onDelete: Cascade)
  mnemonic    String?
  ownSentence String?
  updatedAt   DateTime @updatedAt
  @@unique([userId, wordId])
}
```

6.2 Reglas de integridad (validadas con Zod en el servicio)

- `type = NOUN` ⇒ `gender` obligatorio y (`pluralForm` obligatorio o `noPlural = true`). El `lemma` de un sustantivo empieza con mayúscula.
- `type = VERB` con `separablePrefix` ⇒ `lemma` con barra (`ab|biegen`) y `thirdPerson` obligatorio.
- `type = EXPRESSION` ⇒ `clozeText` obligatorio con exactamente un hueco `{{...}}`.
- `translationEs` no vacío; máximo 5 alternativas separadas por `/ `.
- Los campos de texto se guardan normalizados en Unicode NFC y sin espacios sobrantes.
- Un usuario solo puede editar o borrar palabras de su propio `scope`; las globales se pueden ocultar (suspendiendo sus tarjetas) pero no editar. Para corregir una global, la app ofrece «crear copia propia».

7. Lógica de negocio

7.1 Tipos de tarjeta por palabra

Al introducirse una palabra para un usuario se crean sus tarjetas según esta tabla (solo las activadas en `enabledCardKinds`). La tarjeta **principal** es la primera de cada fila; las demás son **secundarias** y solo pasan a ser elegibles cuando la principal alcanza el estado `REVIEW`.

Tipo de palabra / nivel	Tarjetas (principal primero)	Notas
Sustantivo (A, B)	ES_TO_DE, DE_TO_ES, PLURAL	ES_TO_DE exige artículo + sustantivo («der Bahnhof»).
Verbo (A, B)	ES_TO_DE, DE_TO_ES, VERB_FORMS*, SEPARABLE_ORDER**	*solo si irregular, separable o con auxiliar SEIN. **solo si separable y tiene exampleDe.
Adjetivo, adverbio, preposición, pronombre, otro (A, B)	ES_TO_DE, DE_TO_ES	Preposiciones muestran el caso que rigen.
Expresión (A, B)	CLOZE, DE_TO_ES	El hueco se rellena escribiendo.
Cualquier palabra de nivel C	DE_TO_ES	Solo reconocimiento; se permite revelar sin escribir.

Contenido de cada tarjeta

Kind	Pregunta (anverso)	Respuesta esperada
ES_TO_DE	Traducción española + etiqueta del tipo («sustantivo – con artículo»). Si es falso amigo, NO se avisa en el anverso.	Sustantivo: «der Bahnhof». Verbo: infinitivo sin barra («abbiegen»). Resto: lema.
DE_TO_ES	Lema alemán con color de género y botón de audio.	Cualquiera de las alternativas de `translationEs`; o autocalificación.
PLURAL	«der Bahnhof → die …»	`pluralForm` («Bahnhöfe»); se acepta con o sin «die».
VERB_FORMS	«abbiegen: er … / Perfekt: …»	Dos campos: `thirdPerson` y `perfekt` (incluye auxiliar).
SEPARABLE_ORDER	Palabras de `exampleDe` desordenadas como fichas + traducción.	Orden exacto de la frase original.
CLOZE	`clozeText` con el hueco vacío + `exampleEs`.	Texto dentro de `{{...}}`.

7.2 Prioridad de introducción

La prioridad es un entero calculado en `vocabulary/priority.ts` y guardado en `Word.priority`. Mayor = antes.

```
base        = { A: 300, B: 200, C: 100 }[level]
+ 50  si starred
+ 20  si isFalseFriend
- 20  si isCognate
+ min(sources.length, 4) * 5
+ 10  si frequencyRank != null && frequencyRank <= 1000
Desempate al ordenar: lesson.sortOrder ASC, luego createdAt ASC, luego id ASC.
```

7.3 Día de estudio y zona horaria

El «día de estudio» empieza a las `dayStartsAtHour` (por defecto 04:00) en `timezone` del usuario. `studyDay(now, tz, hour)` devuelve una cadena `YYYY-MM-DD`; `dayBounds(...)` devuelve inicio y fin en

UTC. Todos los contadores diarios (nuevas introducidas hoy, repasos hechos hoy, racha) usan esta función.

7.4 Construcción de la cola diaria (`study/queue.ts`)

```
function buildQueue(input: {
  now: Date; settings: UserSettings; dayStart: Date; dayEnd: Date;
  learningDue: CardWithWord[];    // state LEARNING|RELEARNING, due <= now + 20 min
  reviewDue: CardWithWord[];      // state REVIEW, due < dayEnd
  newCandidates: CardWithWord[];  // state NEW, elegibles (ver reglas), ordenadas por prioridad
  reviewsDoneToday: number; newIntroducedToday: number;
  wordsSeenToday: Set<wordId>;    // para separar hermanas
}): QueueItem[]

1. learning = learningDue ordenadas por due ASC.
2. reviews  = reviewDue  ordenadas por due ASC, excluyendo palabras en wordsSeenToday,
              limitadas a max(0, maxReviewsPerDay - reviewsDoneToday).
3. pendientes = learning.length + reviewDue.length
   nuevasPermitidas = pendientes > reviewBacklogCap ? 0
                    : max(0, newCardsPerDay - newIntroducedToday)
4. nuevas = primeras `nuevasPermitidas` de newCandidates, como máximo UNA tarjeta por palabra,
            excluyendo palabras ya presentes en learning/reviews o en wordsSeenToday.
5. Intercalar: learning primero (si su due ya llegó); después reviews con una nueva
   cada k repasos, k = max(1, floor(reviews.length / max(1, nuevas.length))).
6. Devolver la lista. El cliente pide lotes de 20 (GET /api/study/queue).
```

Elegibilidad de nuevas: una tarjeta principal NEW es elegible si su palabra no está suspendida; una secundaria NEW lo es solo si la principal de la misma palabra está en `REVIEW`. Las tarjetas en LEARNING cuyo `due` aún no llegó se muestran cuando no queda nada más; si no queda nada, la pantalla indica «vuelve en N minutos».

Separación de hermanas: tras responder una tarjeta, cualquier otra tarjeta de la misma palabra con `due` dentro del día actual se omite hoy (no se modifica su `due`; simplemente se filtra por `wordsSeenToday`).

7.5 Programación FSRS (`srs/`)

```
// srs/scheduler.ts — interfaz propia; el resto de la app NO importa ts-fsrs
export interface CardSnapshot {
  state: CardState; due: Date; stability: number; difficulty: number;
  elapsedDays: number; scheduledDays: number; learningSteps: number;
  reps: number; lapses: number; lastReview: Date | null;
}
export interface Scheduler {
  preview(card: CardSnapshot, now: Date): Record<Rating, { due: Date; label: string }>;
  review(card: CardSnapshot, rating: Rating, now: Date): CardSnapshot;
}
export function createScheduler(opts: {
  desiredRetention: number; maximumIntervalDays: number; enableFuzz: boolean;
}): Scheduler
```

- `fsrs-scheduler.ts` implementa la interfaz con `ts-fsrs` (`fsrs(generatorParameters({ request_retention, maximum_interval, enable_fuzz }))`, `createEmptyCard`, `repeat`/`next`). Claude Code DEBE confirmar los nombres exactos en la versión instalada y mapear los campos snake_case ↔ camelCase en un único lugar.
- Parámetros de modelo: los valores por defecto de la librería. No se implementa optimización de parámetros en v1.
- Mapeo de botones: 1 = AGAIN («Otra vez»), 2 = HARD («Difícil»), 3 = GOOD («Bien»), 4 = EASY («Fácil»).
- `label` de vista previa: «<1 min», «10 min», «1 h», «1 d», «3 d», «2 sem», «1 mes», «1,5 meses», «1 año» (redondeo legible, en español).

- Rebelde: tras un repaso con AGAIN desde estado REVIEW, si `lapses >= leechThreshold` ⇒ `isLeech = true`; si `leechAutoSuspend` ⇒ `suspended = true`. Se muestra el aviso «Esta palabra es rebelde: escribe un truco para recordarla».

7.6 Corrección de respuestas (`study/answer-check.ts`)

Función pura compartida por cliente (respuesta inmediata) y servidor (fuente de verdad al guardar).

```
checkAnswer(kind, word, typed, opts: { strictCapitalization: boolean })
  -> { verdict: 'CORRECT' | 'ALMOST' | 'WRONG', expected: string, hints: string[] }

normalizar(s): NFC, trim, espacios múltiples -> uno, quitar . ! ? finales, comillas tipográficas -> rectas.

Reglas para respuestas en ALEMÁN (ES_TO_DE, PLURAL, VERB_FORMS, CLOZE):
 1. Igualdad exacta tras normalizar                          -> CORRECT
 2. Sustantivo: artículo distinto o ausente                 -> WRONG  (hint: "Falta o falla el artículo")
 3. Solo difiere en mayúsculas:
       strictCapitalization=false -> CORRECT + hint "Los sustantivos van con mayúscula"
       strictCapitalization=true  -> ALMOST
 4. Escribió ae/oe/ue/ss en lugar de ä/ö/ü/ß                 -> ALMOST (hint: "Usa ä ö ü ß")
 5. Distancia de Levenshtein (sin artículo):
       <= 1 si la palabra tiene 4..9 letras, <= 2 si tiene >= 10 -> ALMOST
 6. En otro caso                                             -> WRONG
 Verbos ES_TO_DE: se acepta el infinitivo con o sin barra ("abbiegen" / "ab|biegen").
 PLURAL: se acepta con o sin "die". Nunca acepta la forma singular.

Reglas para respuestas en ESPAÑOL (DE_TO_ES):
 - Comparar contra cada alternativa de translationEs, ignorando mayúsculas, tildes,
   artículos iniciales (el/la/los/las/un/una) y el "se" final de verbos pronominales.
 - Coincidencia -> CORRECT; Levenshtein <= 2 -> ALMOST; si no -> WRONG.

Calificación sugerida: CORRECT -> GOOD, ALMOST -> HARD, WRONG -> AGAIN.
El usuario siempre puede elegir otro botón (1-4). Tarjetas DE_TO_ES permiten "Mostrar respuesta"
sin escribir -> verdict SELF_GRADED.
```

La tabla de casos de prueba de la sección 12.2 es parte de la especificación de esta función.

7.7 Registro de un repaso (transacción)

- Validar que la tarjeta pertenece al usuario y no está suspendida.
- Recalcular el veredicto en el servidor con `checkAnswer` (si hubo respuesta escrita).
- Guardar `cardBefore` (instantánea JSON), calcular el nuevo estado con `Scheduler.review`, aplicar reglas de rebelde, fijar `introducedAt` si era NEW.
- Insertar `ReviewLog` con `studyDay`, `isNewCard` y `durationMs` (tope 120 000 ms para no distorsionar estadísticas).
- Devolver el estado nuevo y el siguiente lote si se solicita.

7.8 Deshacer

`POST /api/study/undo` revierte el último `ReviewLog` del usuario del día de estudio actual: restaura la tarjeta desde `cardBefore` y elimina el log, en una transacción. Solo se puede deshacer en cadena hasta 10 repasos atrás. Atajo de teclado: `U`.

7.9 Práctica libre (no afecta FSRS)

- **der/die/das:** muestra sustantivos ya introducidos sin artículo; el usuario pulsa der, die o das (o teclas 1/2/3). Rondas de 20, prioriza los que más falla (contador en memoria de la sesión).

- **Por lección:** recorre todas las palabras de una lección elegida (introducidas o no) en modo tarjeta, útil antes de una clase o examen.
- Ninguna práctica libre crea `ReviewLog` ni modifica `Card`.

8. Interfaz y rutas

8.1 Páginas

Ruta	Contenido y comportamiento
`/login`, `/registro`	Formularios con validación. El registro solo existe si `ALLOW_REGISTRATION=true`. Tras registrarse se crean `UserSettings` por defecto.
`/` (Panel)	Tarjetas resumen: nuevas disponibles hoy, repasos pendientes, en aprendizaje; racha actual; botón grande «Estudiar»; barras de progreso por nivel (A/B/C: introducidas / total, maduras = intervalo ≥ 21 d); mini gráfico de previsión de 7 días.
`/estudiar`	Sesión de estudio (8.2). Barra de progreso de la sesión. Al terminar: resumen (repasadas, aciertos, tiempo, próximas).
`/practicar`	Selector: «der/die/das» o «Por lección» (7.9).
`/vocabulario`	Tabla paginada (50 por página) con búsqueda por alemán o español y filtros: nivel, tipo, lección, género, estado (nueva / aprendiendo / repaso / suspendida / rebelde), falso amigo, propias/globales. Acciones: ver, editar (propias), suspender/reactivar.
`/vocabulario/nueva`	Formulario dinámico según el tipo de palabra; los campos obligatorios cambian (6.2). Vista previa de la tarjeta.
`/vocabulario/[id]`	Ficha completa: datos, color de género, audio, historial de repasos, estado de cada tarjeta, mnemotecnia y «mi frase», botón «crear copia propia» para globales.
`/rebeldes`	Lista de tarjetas con `isLeech`, ordenadas por lapses; edición rápida de mnemotecnia; botón «reiniciar tarjeta» (vuelve a NEW).
`/estadisticas`	Racha actual y máxima; heatmap de 90 días; repasos por día (30 días); retención real (30 días); previsión de 30 días; progreso por nivel y por lección; tiempo medio por tarjeta.
`/importar-exportar`	Subir CSV con vista previa y errores por fila antes de confirmar; exportar CSV, CSV para Anki, copia de seguridad JSON; restaurar copia.
`/ajustes`	Todos los campos de `UserSettings` con rangos validados, voz (lista de voces de-DE disponibles en el navegador), prueba de audio, cambio de contraseña.

8.2 Pantalla de estudio

- **Anverso:** contenido según 7.1; etiqueta de nivel y lección en pequeño.
- **Campo de respuesta** con autofocus, sin autocorrector (`autoComplete="off"`, `spellCheck=false`, `autoCapitalize="off"`, `lang="de"`), y barra de botones `ä ö ü ß Ä Ö Ü` que inserta en la posición del cursor. Atajos: Alt+a/o/u/s.
- **Retroalimentación:** verde (CORRECT), ámbar (ALMOST), rojo (WRONG). Se muestra la respuesta correcta con artículo coloreado, plural, ejemplo DE/ES, aviso de falso amigo, mnemotecnia si existe, y un diff carácter a carácter cuando es ALMOST/WRONG.
- **Botones de calificación** 1–4 con la etiqueta de intervalo de `preview`; el sugerido aparece resaltado y se elige con Enter.
- **Audio:** botón altavoz (tecla `P`); `ttsAutoplay` reproduce al revelar. Usa `speechSynthesis` con `lang = "de-DE"` y la voz elegida en ajustes. Si no hay voz alemana, el botón se oculta y se muestra un aviso una sola vez.
- **Atajos:** Enter = comprobar / confirmar sugerida; 1–4 = calificar; Espacio = revelar (solo reconocimiento); `U` = deshacer; `P` = audio; `E` = abrir ficha en otra pestaña. Lista de atajos con `?`.
- **SEPARABLE_ORDER:** fichas pulsables (y arrastrables) que forman la frase; botón «borrar última».

- El tiempo de respuesta (`durationMs`) se mide desde que aparece el anverso hasta que se califica.

8.3 Diseño

- Responsive desde 360 px; la pantalla de estudio usa una columna centrada de máximo 640 px.
- Modo claro/oscuro según el sistema, con conmutador manual.
- Accesibilidad: contraste AA, foco visible, todos los botones con etiqueta, el color de género SIEMPRE acompañado del texto del artículo.
- Componentes reutilizables: `GenderBadge`, `WordDisplay`, `UmlautBar`, `RatingBar`, `AnswerFeedback`, `StatCard`.

8.4 API JSON (Route Handlers)

Todas exigen sesión salvo `/api/health`. Errores con formato `{ error: { code, message } }` y códigos HTTP 400 (validación), 401, 403, 404, 409, 422, 500. Las mutaciones simples de formularios (palabras, notas, ajustes) se implementan como **Server Actions** con Zod; las rutas siguientes existen porque la pantalla de estudio y la transferencia de archivos las consumen desde el cliente.

Método y ruta	Entrada	Salida
GET `/api/health`	—	`{ status: "ok", db: "ok" }` (hace SELECT 1).
GET `/api/study/summary`	—	`{ newAvailable, reviewsDue, learning, studyDay }`
GET `/api/study/queue?limit=20`	limit 1..50	`{ items: QueueItem[] }` con tarjeta, palabra, nota y preview de intervalos.
POST `/api/study/review`	`{ cardId, rating, typedAnswer?, durationMs }`	`{ verdict, card, isLeech, next?: QueueItem[] }`
POST `/api/study/undo`	—	`{ restoredCardId }` o 409 si no hay nada que deshacer.
GET `/api/practice/gender?limit=20`	—	Sustantivos introducidos sin artículo.
GET `/api/practice/lesson/:number`	—	Palabras de la lección.
POST `/api/transfer/import`	multipart `file` (CSV ≤ 2 MB), `dryRun`	`{ created, updated, errors: [{ row, field, message }] }`
GET `/api/transfer/export?format=csv|anki|json`	—	Descarga del archivo con `Content-Disposition`.
POST `/api/transfer/restore`	multipart `file` (JSON)	Reemplaza los datos del usuario en una transacción tras confirmación.
GET `/api/stats?range=30`	range 7..365	Series para gráficos y totales.

9. Vocabulario: formato, extracción y semilla

9.1 Formato CSV (importación, exportación y semilla)

UTF-8 sin BOM, separador `;`, primera fila con cabeceras exactas. Listas internas (`sources`, `tags`) separadas por `|`; alternativas de traducción separadas por `/` . Booleanos `1`/`0`. Celdas vacías = nulo.

```
lemma;type;gender;plural_suffix;plural_form;no_plural;translation_es;example_de;example_es;cloze_text;
level;starred;goethe_level;frequency_rank;lesson;sources;is_cognate;is_false_friend;false_friend_note;
separable_prefix;third_person;perfekt;auxiliary;is_irregular;governs_case;tags
```

(Una sola línea en el archivo real; aquí se parte por ancho de página.) Valores de `type`: noun, verb, adjective, adverb, preposition, pronoun, expression, other. `gender`: der, die, das. `level`: A, B, C. `auxiliary`: haben, sein. La importación hace upsert por (`scope`, `lemma`, `type`).

Filas de ejemplo (tomadas de los materiales del estudiante)

```
Bahnhof;noun;der;-¨e;Bahnhöfe;0;estación de tren;Das Hotel ist zwischen dem Bahnhof und dem Dom.;
  El hotel está entre la estación y la catedral.;;A;1;A1;;13;KB13|AB13;0;0;;;;;;;;stadt
Ampel;noun;die;-n;Ampeln;0;semáforo;An der Ampel fahren Sie nach links.;En el semáforo gire a la izquierda.;;
  B;0;A2;;13;KB13;0;0;;;;;;;;stadt
Wohnung;noun;die;-en;Wohnungen;0;departamento / vivienda;Ich suche eine Wohnung.;Busco un departamento.;;
  A;1;A1;;14;KB14|AB14;0;0;;;;;;;;wohnen
ab|biegen;verb;;;;0;girar (en una calle);Sie biegen rechts ab.;Usted gira a la derecha.;;A;0;A2;;13;KB13;
  0;0;;ab;biegt ab;ist abgebogen;sein;1;;stadt|trennbar
mögen;verb;;;;0;gustar;Ich mag Pizza.;Me gusta la pizza.;;A;1;A1;;901;CLASE2|KB;0;0;;;mag;
  hat gemocht;haben;1;;modalverb
neben;preposition;;;;0;al lado de;Das Café ist neben dem Dom.;El café está al lado de la catedral.;;
  A;1;A1;;13;KB13|AB13;0;0;;;;;;0;Dat./Akk.;wo
Wie komme ich zum ...?;expression;;;;0;¿Cómo llego a...?;;;Wie {{komme}} ich zum Bahnhof?;A;1;;;13;
  KB13|AB13;0;0;;;;;;0;;stadt
Gymnasium;noun;das;-ien;Gymnasien;0;instituto (secundaria);;;;C;0;A2;;;;0;1;
  No es «gimnasio» (= das Fitnessstudio).;;;;;0;;falscher-freund
```

> Las filas anteriores están partidas en varias líneas solo por el ancho de la página. Son ejemplos de formato; la clasificación definitiva la produce la Fase 0.

9.2 Fase 0 – Extracción del vocabulario desde `materiales/`

Antes de programar, Claude Code DEBE generar `data/vocabulario.csv`, `data/lecciones.csv` y `data/REVISION.md` leyendo los materiales. Particularidades verificadas de los archivos:

- **Kursbuch y Arbeitsbuch** (`*.pdf`): son escaneos con capa de texto OCR ruidosa. Errores típicos observados: «Caf6» = Café, «lch» / «lhr» = Ich / Ihr, «frnde» = finde, «Zimrner» = Zimmer, «m'z» = m², cortes de palabra y letras de casillas mezcladas. Extraer con `pdftotext -layout`, y cuando el texto sea ilegible, renderizar la página a imagen y leerla visualmente.
- **Apuntes de clase** (`M3_Clase_*.docx`, `M4_Clase_*.docx`): a pesar de la extensión, son **texto Markdown en UTF-8**, no archivos Word. Leerlos como texto plano. Contienen gramática (mögen/möchten, verbos separables, posesivos, Perfekt con sein, preguntas personales) con ejemplos y traducciones útiles para `example_de`/`example_es`.
- Temas observados en las lecciones: orientación en la ciudad y pedir direcciones con preposiciones locales + dativo (lección 13), búsqueda de vivienda y habitaciones (lección 14), y el barrio/la ciudad. Confirmar los títulos exactos de cada lección en el propio libro para `lecciones.csv`.
- Los apuntes de clase que no correspondan a una lección del libro se asignan a lecciones virtuales 901, 902… («Clase 2: mögen/möchten», etc.).

Procedimiento

- Extraer el texto de cada fuente y corregir el OCR.
- Lematizar: sustantivos en singular nominativo, verbos en infinitivo (separables con barra), adjetivos en forma base. Unificar duplicados.
- Completar campos obligatorios (6.2): género, plural, traducción al español neutro latinoamericano, ejemplo preferentemente tomado del material.
- Registrar las fuentes en `sources` con los códigos `KB13…KB16`, `AB13…AB16`, `CLASE2`, `CLASE3`, `CLASE4`, `CLASE7`, `CLASE10`.
- Clasificar A/B/C según la tabla de la sección 1 y marcar `starred` si cumple A y B. Si existen las Wortlisten oficiales en `materiales/goethe/`, usarlas para `goethe_level`; si no, dejar `goethe_level` vacío, clasificar por criterio propio y añadir la etiqueta `goethe-sin-verificar`.
- Marcar cognados transparentes y falsos amigos (con nota).
- Excluir nombres propios (personas, Hotel Maritim, Glückstadt), números ya cubiertos y palabras solo de instrucciones del libro (ankreuzen, ergänzen, zuordnen) — estas últimas pueden ir como nivel C con etiqueta `instrucciones`.
- Escribir en `REVISION.md` toda palabra dudosa (OCR ilegible, género incierto, traducción ambigua) para revisión humana antes de cargar la semilla.
- Validar el CSV con el mismo esquema Zod de la importación (`pnpm vocab:validate`). Cero errores es condición de cierre de la Fase 0.

9.3 Semilla (`prisma/seed.ts`)

- Lee `data/lecciones.csv` y `data/vocabulario.csv`, valida con Zod y hace **upsert** en el catálogo global. Recalcula `priority`.
- Es **idempotente**: ejecutarlo varias veces no duplica ni borra nada, y nunca toca tarjetas, logs ni datos de usuarios.
- Si `SEED_DEMO_EMAIL` y `SEED_DEMO_PASSWORD` están definidos y el usuario no existe, lo crea con ajustes por defecto.
- Imprime un resumen: palabras creadas/actualizadas por nivel y lección.

9.4 Exportaciones

- **CSV:** mismo formato de 9.1, con las palabras propias del usuario (y opcionalmente las globales).
- **Anki:** CSV `;` con columnas `Front;Back;Tags`. Front = español; Back = HTML simple con artículo coloreado, plural y ejemplo. Tags = nivel, lección y tipo. Incluye la línea `#separator:semicolon` y `#html:true`.
- **Copia de seguridad JSON:** `{ version: 1, exportedAt, settings, ownWords, cards, reviewLogs, notes }`. La restauración valida la versión y reemplaza los datos del usuario en una transacción, previa confirmación escribiendo «RESTAURAR».

10. Docker y entornos

10.1 Dockerfile (multi-stage)

```
# syntax=docker/dockerfile:1
FROM node:22-alpine AS base
RUN apk add --no-cache libc6-compat openssl && corepack enable
WORKDIR /app

FROM base AS deps
COPY package.json pnpm-lock.yaml ./
COPY prisma ./prisma
RUN pnpm install --frozen-lockfile

FROM base AS build
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN pnpm prisma generate && pnpm build
# esta etapa también se usa como imagen del servicio "migrate"

FROM node:22-alpine AS runner
RUN apk add --no-cache openssl wget
WORKDIR /app
ENV NODE_ENV=production NEXT_TELEMETRY_DISABLED=1 PORT=3000 HOSTNAME=0.0.0.0
RUN addgroup -S app && adduser -S app -G app
COPY --from=build --chown=app:app /app/.next/standalone ./
COPY --from=build --chown=app:app /app/.next/static ./.next/static
COPY --from=build --chown=app:app /app/public ./public
USER app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/api/health || exit 1
CMD ["node", "server.js"]
```

`next.config.ts` DEBE tener `output: "standalone"`. Verificar que el cliente de Prisma generado y su motor para `linux-musl-openssl-3.0.x` quedan incluidos en `.next/standalone`; si no, añadirlos con `outputFileTracingIncludes`.

10.2 docker-compose.yml (producción / uso diario)

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10

  migrate:
    build: { context: ., target: build }
    command: sh -c "pnpm prisma migrate deploy && pnpm db:seed"
    environment:
      DATABASE_URL: ${DATABASE_URL}
      SEED_DEMO_EMAIL: ${SEED_DEMO_EMAIL:-}
      SEED_DEMO_PASSWORD: ${SEED_DEMO_PASSWORD:-}
    depends_on:
      db: { condition: service_healthy }
    restart: "no"

  app:
    build: { context: ., target: runner }
    restart: unless-stopped
    environment:
      DATABASE_URL: ${DATABASE_URL}
```

```
      APP_URL: ${APP_URL}
      ALLOW_REGISTRATION: ${ALLOW_REGISTRATION:-true}
      LOG_LEVEL: ${LOG_LEVEL:-info}
    ports:
      - "${APP_PORT:-3000}:3000"
    depends_on:
      migrate: { condition: service_completed_successfully }

volumes:
  pgdata:
```

10.3 docker-compose.dev.yml y docker-compose.test.yml

```
# dev: solo la base de datos; la app corre con `pnpm dev` en el host (recarga en caliente)
services:
  db:
    image: postgres:16-alpine
    environment: { POSTGRES_USER: deutsch, POSTGRES_PASSWORD: deutsch, POSTGRES_DB: deutsch_dev }
    ports: ["127.0.0.1:5432:5432"]
    volumes: [pgdata_dev:/var/lib/postgresql/data]
volumes: { pgdata_dev: {} }

# test: base efímera para tests de integración y e2e
services:
  db_test:
    image: postgres:16-alpine
    environment: { POSTGRES_USER: test, POSTGRES_PASSWORD: test, POSTGRES_DB: deutsch_test }
    ports: ["127.0.0.1:5433:5432"]
    tmpfs: [/var/lib/postgresql/data]
```

10.4 Variables de entorno (`.env.example`)

```
POSTGRES_USER=deutsch
POSTGRES_PASSWORD=cambia_esta_clave
POSTGRES_DB=deutsch_srs
DATABASE_URL=postgresql://deutsch:cambia_esta_clave@db:5432/deutsch_srs?schema=public
APP_URL=http://localhost:3000
APP_PORT=3000
ALLOW_REGISTRATION=true
LOG_LEVEL=info
SEED_DEMO_EMAIL=
SEED_DEMO_PASSWORD=
```

La app DEBE validar las variables al arrancar con Zod (`src/server/env.ts`) y terminar con un mensaje claro si falta alguna. `.env` está en `.gitignore`; `.dockerignore` excluye `node_modules`, `.next`, `.git`, `tests`, `materiales` y `.env*`.

10.5 Copias de seguridad de la base de datos

```
# scripts/backup.sh  -> backups/deutsch_AAAA-MM-DD_HHMM.sql.gz
docker compose exec -T db pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" \
  | gzip > "backups/deutsch_$(date +%F_%H%M).sql.gz"

# scripts/restore.sh <archivo>  (pide confirmación)
gunzip -c "$1" | docker compose exec -T db psql -U "$POSTGRES_USER" "$POSTGRES_DB"
```

10.6 Scripts de package.json

Script	Acción
`dev`	`next dev` (con `docker-compose.dev.yml` levantado)
`build` / `start`	`next build` / `node .next/standalone/server.js`
`check`	`tsc --noEmit && eslint . && prettier --check .`
`test` / `test:int` / `test:e2e`	Vitest unitarios / integración contra `db_test` / Playwright
`db:migrate` / `db:seed` / `db:studio`	`prisma migrate dev` / `tsx prisma/seed.ts` / `prisma studio`
`vocab:validate`	Valida `data/vocabulario.csv` con el esquema de importación

11. Seguridad

- Contraseñas con Argon2id (parámetros por defecto de `@node-rs/argon2`), mínimo 10 caracteres.
- Sesión: token aleatorio de 32 bytes (base64url) en cookie `wz_session` `HttpOnly`, `SameSite=Lax`, `Secure` cuando `APP_URL` es https, `Path=/`. En base de datos se guarda solo su SHA-256. Duración 30 días, renovación deslizante cuando quedan menos de 15. Logout borra la fila.
- `middleware.ts` redirige a `/login` toda ruta de `(app)` sin cookie; la validación real de la sesión se hace en el servidor en cada acción/consulta (`requireUser()`).
- Límite de intentos de login: 5 por minuto por IP y por email (memoria del proceso; suficiente en un monolito de una instancia). Mensaje de error genérico.
- Toda entrada validada con Zod; todo acceso a datos filtrado por `userId`; las palabras globales son de solo lectura para usuarios.
- Server Actions: Next.js verifica el origen; además, `APP_URL` se usa para `serverActions.allowedOrigins`.
- Cabeceras: `Content-Security-Policy` básica (self), `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin`, `X-Frame-Options: DENY`.
- Subidas: CSV ≤ 2 MB y JSON ≤ 20 MB, tipo comprobado, procesadas en memoria, nunca guardadas en disco.
- Los logs nunca incluyen contraseñas, tokens ni respuestas completas de usuario.

12. Pruebas y criterios de aceptación

12.1 Estrategia

Nivel	Herramienta	Qué cubre	Meta
Unitario	Vitest	answer-check, priority, queue, study-day, streak, csv, adaptador FSRS, schemas	≥ 90 % de líneas en `src/modules/**` puros
Integración	Vitest + `db_test`	Servicios con Prisma: registrar repaso, deshacer, importar, aislamiento entre usuarios	Todos los servicios
E2E	Playwright (Chromium y móvil 390 px)	Flujos de usuario contra la pila de Docker	Flujos de 12.3

12.2 Casos obligatorios de `checkAnswer`

Tarjeta / esperado	Escrito	Veredicto
ES_TO_DE «der Bahnhof»	der Bahnhof	CORRECT
ES_TO_DE «der Bahnhof»	der bahnhof	CORRECT + hint (estricto: ALMOST)
ES_TO_DE «der Bahnhof»	die Bahnhof	WRONG (artículo)
ES_TO_DE «der Bahnhof»	Bahnhof	WRONG (falta artículo)
ES_TO_DE «der Bahnhof»	der Banhof	ALMOST (Levenshtein 1)
ES_TO_DE «die Straße»	die Strasse	ALMOST (ß)
ES_TO_DE «die Brücke»	die Bruecke	ALMOST (ü)
ES_TO_DE «abbiegen»	ab|biegen	CORRECT
PLURAL «Bahnhöfe»	die Bahnhöfe	CORRECT
PLURAL «Bahnhöfe»	Bahnhof	WRONG (singular)
DE_TO_ES «departamento / vivienda»	La vivienda	CORRECT
DE_TO_ES «semáforo»	semaforo	CORRECT (sin tilde)
DE_TO_ES «girar (en una calle)»	girar	CORRECT (se ignora el paréntesis)
VERB_FORMS «biegt ab / ist abgebogen»	biegt ab / hat abgebogen	WRONG en Perfekt (auxiliar)

Regla añadida por la tabla: el texto entre paréntesis de `translationEs` es una aclaración y no se exige en la respuesta.

12.3 Criterios de aceptación (Dado / Cuando / Entonces)

- **Arranque:** dado un clon limpio con `.env` copiado de `.env.example`, cuando se ejecuta `docker compose up --build`, entonces en menos de 5 minutos `/api/health` responde 200 y el catálogo tiene las palabras del CSV.
- **Persistencia:** dado un usuario con repasos, cuando se ejecuta `docker compose down` y luego `up`, entonces los datos siguen ahí (`down -v` sí los borra, documentado en README).
- **Límite de nuevas:** dado `newCardsPerDay = 12`, cuando el usuario estudia todo, entonces se introducen exactamente 12 palabras nuevas ese día de estudio y el panel muestra 0 nuevas disponibles.

- **Tope por acumulación:** dado 130 repasos pendientes y `reviewBacklogCap = 120`, entonces la cola no contiene tarjetas nuevas.
- **Orden:** dadas palabras A★, A, B y C nuevas, entonces se introducen en ese orden.
- **Hermanas:** después de responder la tarjeta ES_TO_DE de una palabra, ninguna otra tarjeta de esa palabra aparece el mismo día.
- **Secundarias:** la tarjeta PLURAL de un sustantivo no aparece hasta que su ES_TO_DE está en REVIEW.
- **Deshacer:** tras calificar y pulsar `U`, la tarjeta vuelve exactamente a su estado anterior y el log desaparece.
- **Rebelde:** al llegar a 6 lapses, la tarjeta aparece en `/rebeldes` y se muestra el aviso con campo de mnemotecnia.
- **Aislamiento:** el usuario B nunca puede leer ni modificar tarjetas, notas o palabras propias del usuario A (prueba de integración que intenta acceder por id).
- **Importación:** un CSV con 2 filas inválidas muestra en la vista previa los errores con número de fila y campo, y no guarda nada hasta confirmar las filas válidas.
- **Día de estudio:** un repaso a las 02:00 hora de Bolivia cuenta para el día anterior (inicio a las 04:00).
- **Racha:** la racha cuenta días de estudio consecutivos con al menos un repaso; un día sin repasos la reinicia.
- **Móvil:** en 390 px de ancho, la sesión de estudio completa se puede hacer sin desplazamiento horizontal.

13. Plan de implementación por fases

Claude Code DEBE trabajar fase por fase, en orden. Cada fase termina con `pnpm check` y las pruebas en verde, un commit con mensaje convencional (`feat:`, `fix:`, `test:`…) y una nota breve en `docs/DECISIONES.md` si se tomó alguna decisión no prevista aquí.

Fase	Entregables	Hecho cuando…
0. Vocabulario	`data/vocabulario.csv`, `data/lecciones.csv`, `data/REVISION.md`, script `vocab:validate`	El CSV valida sin errores y `REVISION.md` lista las dudas. **Pausa: el estudiante revisa antes de seguir.**
1. Esqueleto	Next.js + TS strict, Tailwind, shadcn/ui, ESLint/Prettier, `env.ts`, logger, Dockerfile, los tres compose, `/api/health`	`docker compose up --build` levanta todo y health = 200.
2. Datos	`schema.prisma`, primera migración, `seed.ts` idempotente, repositorios base	La semilla carga el CSV; ejecutarla dos veces no cambia nada.
3. Autenticación	Registro, login, logout, sesiones, middleware, límite de intentos, ajustes por defecto	Pruebas de integración de sesión y aislamiento en verde.
4. Motor	`priority`, `study-day`, `queue`, `answer-check`, adaptador FSRS, leech, servicio de repaso y deshacer	Casos 12.2 y criterios de cola/hermanas/secundarias en verde (unit + integración).
5. Estudio	Pantalla `/estudiar`, API de estudio, UmlautBar, RatingBar, feedback con diff, atajos, TTS	E2E: estudiar 5 tarjetas, deshacer 1, ver resumen.
6. Vocabulario	Listado con filtros, ficha, formulario dinámico, copia propia, notas, `/rebeldes`	Crear un sustantivo sin género falla con mensaje claro; con género funciona y genera sus tarjetas.
7. Transferencia	Importación con vista previa, exportación CSV/Anki/JSON, restauración	Exportar → borrar → restaurar devuelve el mismo estado.
8. Estadísticas	Panel y `/estadisticas` con todas las series	Datos de prueba fijos producen los valores esperados.
9. Práctica y ajustes	`/practicar` (der/die/das, por lección), `/ajustes` completo	La práctica no crea `ReviewLog` (prueba de integración).
10. Cierre	Revisión de accesibilidad, modo oscuro, pruebas e2e móviles, README completo	Todos los criterios de 12.3 en verde.

14. CLAUDE.md (copiar en la raíz del repositorio)

```
# Wortschatz SRS — guía para Claude Code

## Fuente de verdad
- La especificación es docs/ESPECIFICACION.pdf. Si algo no está claro, elige la opción más
  simple compatible con ella y regístrala en docs/DECISIONES.md. No inventes funciones fuera
  del alcance de la versión 1.

## Arquitectura (no negociable)
- Monolito modular en una sola app Next.js. Nada de microservicios, Redis ni colas.
- UI -> módulos (src/modules/*/index.ts) -> infraestructura (src/server). Nunca al revés.
- Componentes y páginas NO importan Prisma. Todo repositorio recibe userId primero.
- Lógica pura sin E/S y con `now` inyectado. ts-fsrs solo se importa en srs/fsrs-scheduler.ts.
- Operaciones multi-tabla dentro de prisma.$transaction.

## Convenciones
- Código, identificadores y BD en inglés; UI, errores al usuario y docs en español.
- TypeScript strict, sin `any` implícito. Validación con Zod en todas las entradas.
- Commits pequeños y convencionales. Una fase = una rama o un bloque de commits.
- Cambios de esquema SIEMPRE con `pnpm db:migrate` (nunca editar migraciones aplicadas).

## Antes de dar una fase por terminada
1. pnpm check
2. pnpm test && pnpm test:int (con docker compose -f docker-compose.test.yml up -d)
3. pnpm test:e2e cuando la fase toca UI
4. docker compose up --build y comprobar /api/health

## Vocabulario
- Los .docx de materiales/clases son Markdown: léelos como texto.
- Los PDF tienen OCR ruidoso: corrige (Caf6 -> Café, lch -> Ich) y, si dudas, mira la página como imagen.
- Nunca guardes un sustantivo sin género. Anota toda duda en data/REVISION.md.

## Pedagogía (no romper)
- Producción (español -> alemán) exige escribir antes de ver la respuesta.
- Orden de nuevas: A★ -> A -> B -> C. Máx. 1 tarjeta por palabra al día.
- Colores de género: der #2563EB, die #DC2626, das #16A34A, siempre junto al artículo escrito.
```

15. Evolución futura (fuera de la versión 1)

- Optimización de parámetros FSRS con el historial propio del usuario (cuando haya ≥ 1 000 repasos).
- PWA con modo offline y sincronización al reconectar.
- Audio grabado o voces neuronales generadas en el servidor y cacheadas.
- Ejercicios de dictado y de producción libre corregidos con un modelo de lenguaje.
- Exportación nativa `.apkg` de Anki.
- Nuevas lecciones A2 añadidas importando más CSV con el mismo formato.

Supuesto de base: los valores numéricos por defecto (12 nuevas al día, tope de 120, retención 0,90, umbral de rebelde 6, inicio de día a las 04:00) son puntos de partida razonables, no resultados exactos de investigación. Todos son configurables en `/ajustes` precisamente para ajustarlos con la experiencia real.